

HISABDO AI

DAY 12 — AI ANOMALY EXPLANATION LAYER

Professional MVP Specification & .NET/C# Implementation Guide

1. Purpose

This module converts verified anomaly-detection results from the HisabDo .NET backend into simple, understandable messages for the user. The AI explanation layer does not detect anomalies or calculate financial values; it only explains verified backend results.

2. Source and Architecture Rule

The Financial Health specification establishes the same core rule: financial values must come from trusted HisabDo data and a deterministic calculation/analytics layer, while the LLM converts verified values into a clear explanation.

Correct flow:

HisabDo Database → .NET Anomaly Detection Engine → Verified Anomaly Results → AI Explanation Layer → User

3. Omesha — AI Explanation Tasks

- Create explanation structure.
- Convert verified anomalies into understandable messages.
- Generate severity/explanation text.
- Ensure AI does not invent transaction values.
- Prepare sample AI responses.

4. Explanation Structure

Every anomaly explanation should follow this simple structure:

1. What happened — state the verified anomaly type.
2. Why it was flagged — briefly explain the comparison or rule used.
3. Severity — use the backend-provided severity; AI must not change it.
4. Relevant verified values — use only values supplied by the backend.
5. What the user can do — provide a practical, non-accusatory suggestion.

5. Supported Anomaly Types

Anomaly Type	Explanation Focus	Example
--------------	-------------------	---------

Unusual Amount	Single expense is much higher than the user's normal category pattern.	A grocery expense was significantly above the verified historical category baseline.
Sudden Spending Spike	Current category spending increased sharply compared with the comparable historical period.	Dining spending increased by the verified percentage.
Unusual Category	A category shows spending materially above its historical baseline.	A category is above the verified 2× historical baseline.
Duplicate Transaction	Two or more transactions match the backend duplicate-candidate rules.	Two same-day transactions have matching verified details.

6. Severity Explanation Rules

Severity must be calculated by the backend anomaly engine and passed to the AI as a verified field. The AI only explains it.

- Low — minor deviation; explain calmly and avoid alarming language.
- Medium — meaningful deviation; explain the pattern and suggest reviewing the expense.
- High — significant deviation; clearly explain the verified reason and recommend checking the transaction.

The AI must never claim that an anomaly is fraud, theft, or an unauthorized transaction unless a separate trusted system explicitly provides that determination.

7. Verified Input Contract to AI

Recommended DTO:

```
public sealed record AnomalyExplanationInput(
    string AnomalyType,
    string Severity,
    string Category,
    decimal? TransactionAmount,
    decimal? HistoricalAverage,
    decimal? ChangePercentage,
    string? TransactionDate,
    string? Description,
    string DetectionReason
);
```

Only fields that are actually available and verified should be populated. Missing values should remain null rather than being guessed by the AI.

8. AI Safety / No-Invention Rules

- Do not calculate a new transaction amount.
- Do not change backend-provided amounts, percentages, dates, categories, or severity.
- Do not create transaction IDs or dates that were not supplied.
- Do not assume an anomaly is fraud.
- Do not invent a historical average.
- Do not claim certainty when the backend marks an item as a candidate.
- If a required value is missing, explain the anomaly without inventing the missing value.

9. Recommended Prompt Structure

System instruction:

You are the HisabDo financial explanation assistant.

Explain only the verified anomaly data supplied by the backend.

Never invent, modify, recalculate, or estimate financial values.

Do not label an anomaly as fraud.

Use simple, helpful language.

If a value is missing, do not guess it.

Backend input:

Anomaly Type: {AnomalyType}

Severity: {Severity}

Category: {Category}

Transaction Amount: {TransactionAmount}

Historical Average: {HistoricalAverage}

Change Percentage: {ChangePercentage}

Transaction Date: {TransactionDate}

Detection Reason: {DetectionReason}

Output:

1. What happened

2. Why it was flagged

3. Severity

4. Suggested action

10. Sample AI Responses

Unusual Amount — High

This expense was flagged because its amount is significantly higher than your usual spending pattern for this category. Severity: High. Please review the transaction to confirm it is correct.

Sudden Spending Spike — Medium

Your spending in this category has increased noticeably compared with the previous comparable period. Severity: Medium. Reviewing recent purchases in this category may help you understand the increase.

Unusual Category — Medium

This category shows spending above its normal historical pattern. Severity: Medium. Consider reviewing the recent transactions in this category and checking whether the increase was planned.

Duplicate Transaction — High

Two transactions were flagged as possible duplicates because they match the backend's duplicate-checking criteria. Severity: High. Please review both transactions before taking action.

11. .NET/C# Explanation Service

The following reference implementation keeps the explanation rules deterministic and safe. It can be used before connecting an actual LLM provider. The LLM can later receive the same verified DTO.

```
public sealed record AnomalyExplanationInput(
    string AnomalyType,
    string Severity,
    string Category,
    decimal? TransactionAmount,
    decimal? HistoricalAverage,
    decimal? ChangePercentage,
    DateTime? TransactionDate,
    string? Description,
    string DetectionReason
);
```

```
public sealed record AnomalyExplanation(
    string Title,
    string WhatHappened,
    string WhyFlagged,
    string SeverityText,
    string SuggestedAction
);
```

```

public sealed class AnomalyExplanationService
{
    public AnomalyExplanation
    CreateExplanation(AnomalyExplanationInput input)
    {
        ArgumentNullException.ThrowIfNull(input);

        var title = input.AnomalyType switch
        {
            "Unusual Amount" => $"Unusual {input.Category} expense",
            "Sudden Spending Spike" => $"Spending spike in
{input.Category}",
            "Unusual Category" => $"Unusual spending in
{input.Category}",
            "Duplicate Transaction" => "Possible duplicate transaction",
            _ => "Spending anomaly detected"
        };

        var what = input.AnomalyType switch
        {
            "Unusual Amount" =>
                "This expense was flagged because its amount is outside
the user's normal spending pattern.",
            "Sudden Spending Spike" =>
                "Spending in this category increased compared with the
comparable historical period.",
            "Unusual Category" =>
                "This category shows spending above its normal
historical pattern.",
            "Duplicate Transaction" =>
                "This transaction was flagged as a possible duplicate
based on verified transaction details.",
            _ => "The backend detected an unusual spending pattern."
        };

        var why = string.IsNullOrEmpty(input.DetectionReason)
            ? "The anomaly detection engine identified a deviation from
the verified spending baseline."
            : input.DetectionReason;

        var severityText = input.Severity switch
        {
            "High" => "High severity: please review this transaction
carefully.",
            "Medium" => "Medium severity: this spending pattern may
need attention.",
        };
    }
}

```

```

        "Low" => "Low severity: this is a minor deviation from the
normal pattern.",
        _ => $"Severity: {input.Severity}."
    };

    var action = input.AnomalyType == "Duplicate Transaction"
        ? "Review the flagged transactions and confirm whether
both should remain."
        : "Review the transaction and confirm whether the spending
was expected.";

    return new AnomalyExplanation(
        title,
        what,
        why,
        severityText,
        action
    );
}
}

```

12. Optional LLM Integration Contract

After the deterministic service produces the verified explanation input, an LLM provider can be called from the .NET backend through an SDK or HTTP API. The LLM should receive only the approved fields and should return explanation text. The backend remains the source of truth.

13. Testing Requirements

- Correct anomaly type is displayed.
- Backend severity is preserved exactly.
- Verified amount is reproduced without modification when supplied.
- No amount is invented when amount is null.
- No historical average is invented when unavailable.
- Duplicate candidates are described as possible duplicates, not confirmed fraud.
- Prompt injection inside transaction descriptions cannot override system rules.
- Multiple users' anomaly data cannot be mixed.
- Empty/no-anomaly result produces a safe response.

14. Team Handoff

Omesha — AI/ML & Explanation Logic:

- Finalize explanation structure and wording rules.

- Prepare sample verified anomaly inputs and expected outputs.
- Validate severity wording.
- Define prompt safety rules.
- Review AI responses for hallucinated values.

Jaffer — Backend:

- Provide verified anomaly DTO/results from the .NET detection engine.
- Keep calculations and anomaly decisions outside the LLM.
- Pass only user-authorized anomaly data to the explanation layer.

15. Team Lead Acceptance Checklist

- ☐ Explanation structure is approved.
- ☐ All four anomaly types are covered.
- ☐ Severity comes from verified backend logic.
- ☐ AI cannot invent financial values.
- ☐ Sample responses are approved.
- ☐ No fraud/unauthorized claims are made without trusted evidence.
- ☐ Missing values are handled safely.
- ☐ User data isolation is maintained.
- ☐ The .NET implementation is ready to connect to the actual HisabDo backend.

16. Final Definition

The HisabDo AI Anomaly Explanation Layer is a controlled AI communication layer that converts verified anomaly results into simple user-facing explanations. The .NET backend remains responsible for transaction data, anomaly detection, thresholds, and severity. AI is responsible only for understandable language and actionable explanation, without inventing or changing financial facts.